In previous sections, you learned about the functional programming paradigm and how it fits in the Spring ecosystem, using Spring Cloud Function and Spring Cloud Stream. This last section will guide you through the implementation of producers and consumers.

As you'll see, consumers are not that different from the functions you wrote in Dispatcher Service. On the other hand, producers are slightly different because, unlike functions and consumers, they are not naturally activated. I'll show you how to use them both in Order Service while implementing the last part of the order flow for the Polar Bookshop system.

10.5.1 Implementing event consumers, and the problem of idempotency

The Dispatcher Service application we previously built produces messages when orders are dispatched. The Order Service should be notified when that happens so that it can update the order status in the database.

First, open your Order Service project (order-service), and add the dependencies on Spring Cloud Stream and the test binder in the build.gradle file. Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 10.17 Adding dependency for Spring Cloud Stream and test binder

```
dependencies {
  ...
  implementation 'org.springframework.cloud:
  ➥ spring-cloud-stream-binder-rabbit'
  testImplementation("org.springframework.cloud:spring-cloud-stream") {
    artifact {
      name = "spring-cloud-stream"
      extension = "jar"
      type ="test-jar"
```